

Prefix Peeking: Eviction Policy and Scheduling Method for LLM Serving with Long Shared Contexts

Chen Yu Ko and Po Wei Huang

Department of Computer Science and Information Engineering,
National Taiwan University, Taipei, Taiwan

Abstract—Large language model (LLM) serving systems use KV cache prefix reuse to amortise the prefill cost of long shared contexts, but the default LRU eviction policy and FCFS scheduler are oblivious to the prefix needs of pending requests, causing preventable cache misses that inflate latency. We present two complementary, opt-in mechanisms implemented in the vLLM v1 engine and compatible with the Jenga [1] SOSP’25 heterogeneous KV cache allocator (the mechanisms operate at the scheduling layer and are orthogonal to allocator placement): *Prefix Peeking Eviction (PPE)* and *Prefix Peeking Scheduling (PPS)*. PPE marks free KV cache blocks that are needed by queued requests as *protected* before each scheduling step, preventing LRU from evicting blocks that will be immediately reused. PPS re-orders the waiting queue by estimated prefix hit length with an anti-starvation guarantee, so that cache-warm requests are served first. Both mechanisms are strictly read-only with respect to block reference counts and impose bounded overhead. On an arXiv QA benchmark with 90k-token shared contexts (10 articles $\times$ 4 questions, dataset: liyucheng/arxiv-march-2023), combining both mechanisms achieves $2.12\times$ higher throughput and 49% lower mean TTFT over LRU on Gemma-3-4B-IT (mean across $n = 4$ LRU runs and $n = 2$ combined runs; the LRU baseline shows run-to-run standard deviation of 6.4%, reflecting GPU thermal variance at this low request rate). We characterise a pool-size-to-article-size ratio ρ that governs when PPE is beneficial; $\rho_{\min} = 2.0$ is an empirically derived operating boundary across three distinct pool regimes, pending validation at intermediate ratios. PPS provides consistent throughput gains across all pool-size regimes. A Poisson rate-sweep experiment over Gemma-3-4B-IT confirms the advantage holds across every tested rate (0.2–4.0 req/s, single run per rate), with Both outperforming LRU by 25–81% in throughput and 20–46% in E2E latency. A control experiment on MMLU-pro (no shared prefix) confirms $< 0.6\%$ throughput variance, ruling out regression.

Index Terms—LLM serving, KV cache management, prefix caching, eviction policy, request scheduling, time-to-first-token

I. INTRODUCTION

Transformer-based large language models are increasingly deployed in serving settings where multiple requests share a long common prefix—a retrieved academic article, a lengthy system prompt, or a multi-turn conversation history. *Prefix caching* [2] amortises the quadratic prefill cost by retaining the key-value (KV) activations of previously processed tokens: when a new request matches a cached prefix, the server skips prefill for those tokens and proceeds directly to autoregressive decoding.

Despite this mechanism, two systemic inefficiencies persist in stock vLLM [2] and related systems:

- 1) **Eviction blindness.** The LRU eviction policy has no visibility into the waiting queue. Under cache pressure it may discard a block whose content will be needed by the very next request to be scheduled, forcing a full re-prefill that could have been avoided.
- 2) **Scheduling blindness.** The default FCFS scheduler treats all waiting requests equally regardless of their prefix hit potential. A cold request (no cached blocks) may be scheduled ahead of a warm one (many cached blocks), triggering eviction of the warm blocks to make room for the cold prefill and wasting GPU cycles.

Both pathologies are most acute in long-context workloads where each article occupies a substantial fraction of the KV cache pool, and where the pool holds only a small number of articles simultaneously.

Practical visibility of the waiting queue. In a single-node LLM serving process, the scheduler and the KV cache manager share the same address space and the same in-process data structures. The lifecycle of a request is as follows: (1) a client request arrives at the HTTP front-end and is appended to the waiting queue; (2) the scheduler reads the waiting queue at each engine step to select the next batch; (3) the KV cache allocator assigns blocks and launches the prefill kernel; (4) autoregressive decoding proceeds; and (5) the completed response is returned. Steps (1)–(2) reveal that the waiting queue is already a live, in-process object that the scheduler reads unconditionally at every step. Both PPE and PPS merely *reuse* this already-visible queue to inform eviction and ordering decisions; they introduce no inter-process communication, no additional memory copies, and no external lookups. This design is therefore deployable in any production single-node vLLM instance without architectural changes.

This paper presents two complementary mechanisms that make the eviction and scheduling subsystems *demand-aware*:

- **Prefix Peeking Eviction (PPE):** Before each scheduling step, inspect the waiting queue’s prefix hash chains and mark matching free blocks as *protected*. The eviction loop skips protected blocks, preserving cache content for imminent requests.
- **Prefix Peeking Scheduling (PPS):** Reorder the waiting queue by descending estimated prefix-hit token count, with an anti-starvation guarantee, so that the warmest requests are dispatched first.

Both mechanisms are implemented in the vLLM v1 engine. They operate at the scheduling layer and are orthogonal to the KV cache allocator, so they compose with Jenga [1], a SOSP’25 system for heterogeneous KV cache management, as well as with the stock vLLM allocator used in our evaluation.

We evaluate on an arXiv QA benchmark across three models on an NVIDIA A5000 GPU. Key findings:

- Combined, PPE+PPS achieves a mean of $2.12\times$ throughput and 49% lower mean TTFT over LRU on Gemma-3-4B-IT ($\rho \approx 5$, $n = 2-4$ runs).
- PPE scales with the pool-size-to-article-size ratio ρ ; below the operating-regime boundary $\rho_{\min} = 2.0$ it is automatically disabled.
- PPS provides consistent mean gains in all three pool-size regimes tested (+9% to +37%).
- A Poisson rate-sweep experiment on Gemma-3-4B-IT (single run per rate) shows Both outperforming LRU by 25–81% in throughput across all 20 tested rates.
- A MMLU-pro control experiment (no shared prefix) confirms $< 0.6\%$ throughput variance, ruling out regression.

The remainder of this paper is organised as follows. Section II reviews Jenga and vLLM prefix caching. Sections III and IV describe PPE and PPS. Section V presents experimental results. Section VI surveys related work, and Section VII concludes.

II. BACKGROUND

A. Jenga: Heterogeneous KV Cache Management

Jenga [1] addresses the observation that different transformer layers exhibit distinct access patterns [1]: early layers that capture shared context prefixes are reused across many requests, while later layers are accessed once per generation step. Jenga introduces a two-level page table that maps logical KV blocks to physical locations across GPU HBM, CPU DRAM, and NVMe SSDs, enabling fine-grained placement and migration decisions at block-layer granularity.

Jenga provides three allocator policies: a *static partition* that fixes the GPU/CPU allocation ratio at startup; a *max-page* policy that greedily maximises GPU-resident blocks; and the *Jenga* policy, which adapts placement based on per-layer access frequency and migration cost.

In the original evaluation, Jenga achieves $1.39\times$ – $5.90\times$ higher throughput than the vLLM baseline across models and GPU types (H100 and L4), as shown in Figure 14 of [1]. Figure 15 of [1] shows that for the Llama Vision model under varying request rates, Jenga reduces TTFT by approximately $3-4\times$ at the same load.

Our work extends Jenga’s vLLM v1 engine. The mechanisms described in this paper operate at the scheduling layer and are compatible with all three Jenga allocator policies.

B. vLLM Prefix Caching

vLLM implements prefix caching via a block-level hash table. Each KV cache block stores activations for a fixed token window. A block is identified by the hash of the complete token sequence that contributed to it, including all ancestor blocks in the prefix chain.

When a new request arrives, the scheduler walks the hash table to find the longest matching prefix chain (cache hit), reusing matched blocks without recomputation and allocating fresh blocks only for unmatched tokens. Free blocks—those with zero reference counts—are maintained in an LRU list and evicted under pressure.

The fundamental limitation is that the LRU list has no visibility into the waiting queue: it cannot distinguish a block needed by the next scheduled request from one that will not be reused for many seconds. This motivates PPE (Section III). Similarly, the default FCFS ordering provides no preference to requests that could exploit existing cache content, motivating PPS (Section IV).

C. The Pool-Size-to-Article-Size Ratio

We define the *pool ratio* ρ :

$$\rho = \frac{N_{\text{total}}}{B_{\text{avg}}} \quad (1)$$

where N_{total} is the total GPU KV cache block count and B_{avg} is the average block count required for one article’s prefix. When $\rho \approx 1$, the pool holds only one article, and any new prefill displaces the current article regardless of eviction policy. Our experiments at $\rho \in \{1.1, 2, 5\}$ suggest that eviction protection becomes beneficial at $\rho \geq 2$; we treat this as an operating-regime boundary rather than a sharp threshold, as intermediate values ($\rho \in \{1.5, 3, 4\}$) have not yet been evaluated.

III. PREFIX PEEKING EVICTION

A. Mechanism

Before each scheduling step, PPE scans the waiting queue and identifies free KV cache blocks that are reachable from the prefix hash chains of queued requests. Those blocks are added to a *protected set*. The LRU eviction loop then skips any block in the protected set, preserving cache content that will be consumed by the immediately upcoming requests.

Without this mechanism, LRU is oblivious to what is waiting: it evicts blocks in recency order, potentially discarding an article that the very next scheduled request will need, thereby forcing a full re-prefill. With PPE, articles for queued requests survive in cache until those requests are scheduled, increasing the effective prefix cache hit rate.

Algorithm 1 gives the pseudocode.

B. Budget Cap

Without a bound on the protected set, a single large article could fill it entirely, leaving no evictable candidates when fresh allocation is needed. We cap the number of protected blocks at half of the current free pool:

$$\text{budget} = \max(1, \lfloor N_{\text{free}}/2 \rfloor) \quad (2)$$

This guarantees that at least 50% of free blocks remain evictable at all times. The standard LRU eviction loop walks the free list until it collects the required number of evictable blocks; protected blocks are skipped. Because at most half the

Algorithm 1 Prefix Peeking Eviction (PPE)

Require: Waiting queue Q ; free block list F ; total block count N_{total}

- 1: $B_{\text{avg}} \leftarrow \text{mean blocks}(\text{first } \min(5, |Q|) \text{ requests in } Q)$
- 2: $\rho \leftarrow N_{\text{total}} / B_{\text{avg}}$
- 3: **if** $\rho < \rho_{\min}$ **then return** ▷ auto-disabled; fall back to pure LRU
- 4: **end if**
- 5: $\text{budget} \leftarrow \max(1, \lfloor |F|/2 \rfloor)$
- 6: $P \leftarrow \emptyset$ ▷ protected set
- 7: **for** each request r in Q **do**
- 8: **for** each free block b reachable from r 's prefix hash chain **do**
- 9: **if** $|P| < \text{budget}$ **then**
- 10: $P \leftarrow P \cup \{b\}$
- 11: **end if**
- 12: **end for**
- 13: **end for**
- 14: **LRU eviction loop:** skip any block $b \in P$

free blocks may be protected, the loop inspects at most $2k$ entries to evict k blocks in the worst case, so the amortised number of entries scanned per evicted block is bounded by a constant (≤ 2) independent of the protected-set size. This worst-case bound holds regardless of how protected blocks are positioned within the list, and keeps the amortised eviction cost $O(1)$. Protection is applied greedily across the waiting queue; requests whose blocks would exceed the budget are left unprotected. The 50% bound is a conservative choice; a sensitivity study varying the cap from 25% to 75% is left for future work.

C. Pool-Size Auto-Disable

When $\rho < \rho_{\min} = 2.0$, any new prefill will necessarily displace the current article because the pool cannot simultaneously hold two articles. Protecting blocks provides no benefit in this regime.

The check is evaluated each scheduling step using the first five waiting requests as an estimate of B_{avg} . If $\rho < \rho_{\min}$, the protected set is cleared and the system falls back to pure LRU with zero overhead.

Example. Llama-3.1-8B on an A5000 GPU with 50k-token contexts has $N_{\text{total}} \approx 3,206$ blocks and $B_{\text{avg}} \approx 2,907$ blocks, giving $\rho \approx 1.10 < 2.0$. The auto-disable fires and the system runs unmodified LRU.

The threshold $\rho_{\min} = 2.0$ is empirically determined from the three model configurations evaluated; an analytical derivation based on eviction miss-rate models is left for future work. The five-request sample is a practical approximation; for highly heterogeneous workloads, a running exponential moving average of request sizes would be more robust.

D. Correctness

Protection never modifies block state. Protected blocks retain zero reference count and remain fully allocatable by the normal path. The protected set is recomputed from scratch each step, so stale protection is impossible.

IV. PREFIX PEEKING SCHEDULING

A. Mechanism

PPS replaces the default FCFS request waiting queue with a priority queue that orders requests by their estimated prefix cache hit length. Cache-warm requests—those with many matching blocks already in the GPU pool—are promoted ahead of cold requests, maximising cache reuse within each scheduling step.

The sort key per request (lower value = higher priority) is:

$$k(r) = \left(\mathbf{1}[t_{\text{wait}}(r) < \tau], \right. \\ \left. -\hat{h}(r), \right. \\ \left. t_{\text{enqueue}}(r) \right) \quad (3)$$

where $t_{\text{wait}}(r)$ is the elapsed wait time, τ is the anti-starvation threshold (default 5 s), $\hat{h}(r)$ is the read-only estimated prefix-hit token count, and $t_{\text{enqueue}}(r)$ is the FCFS tiebreak.

The sort is triggered only when a new request joins the queue, so that repeated dequeue operations within the same scheduling step are $O(1)$.

B. Anti-Starvation

The first component of $k(\cdot)$ places requests that have been waiting $\geq \tau$ seconds in priority band 0, ahead of all non-stale requests regardless of their prefix score. This prevents cold requests from being indefinitely deferred behind a stream of warm arrivals. The threshold τ is configurable; the 5 s default is conservative and suited to the batch-serving workloads evaluated here.

C. Read-Only Prefix Estimation

The prefix hit estimate is computed by walking the block pool hash table in read-only mode—the same path used during actual allocation—with no side effects: no LRU updates, no reference count changes, and no block state mutations. This ensures the estimate does not perturb the eviction order maintained by PPE.

D. Interaction with Preemption

Preempted requests are re-inserted into the priority queue with their original enqueue timestamp preserved, so elapsed wait time accumulates continuously. Preempted requests benefit from anti-starvation on the same footing as newly arrived ones, with no special-case handling.

E. Complementarity with PPE

PPE and PPS address complementary aspects of the same problem:

- PPE preserves cache blocks *between* scheduling steps—temporal protection of the free pool.
- PPS maximises block consumption *within* a scheduling step—spatial prioritisation of warm requests.

When both are active, PPE keeps warm blocks alive long enough for PPS to direct the right requests to them, amplifying the individual gains of either mechanism alone.

TABLE I
GEMMA-3-4B-IT — ARXIV QA (90K AVG. CONTEXT, $\rho \approx 5$). VALUES ARE MEAN $\pm$ STD; n = NUMBER OF INDEPENDENT RUNS.

Policy (n)	req/s	Δ	TTFT _{mean} (s)	TTFT _{p99} (s)
LRU (4)	0.126 $\pm$.008	—	152 $\pm$ 14	314 $\pm$ 19
PPS only (2)	0.173 $\pm$.003	+37%	100 $\pm$ 2	227 $\pm$ 4
PPE only (2)	0.219 $\pm$.009	+74%	94 $\pm$ 2	179 $\pm$ 9
Both (2)	0.267 $\pm$.003	+112%	77 $\pm$ 1	145 $\pm$ 3

TABLE II
MINISTRAL-8B FP8 — ARXIV QA (32K AVG. CONTEXT, $\rho \approx 2$). SINGLE RUN ($n = 1$); VARIANCE DATA PENDING.

Policy	req/s	Δ	TTFT _{mean} (s)	TTFT _{p99} (s)
LRU	0.1725	—	121.8	221.1
PPE only	0.1766	+2.4%	120.3	216.8
PPS only	0.2044	+18.5%	91.8	184.6
Both	0.2193	+27.1%	87.6	170.2

V. EXPERIMENTAL EVALUATION

A. Setup

Hardware. NVIDIA A5000 GPU (24 GB VRAM), AMD EPYC server CPU, 256 GB DDR4 DRAM.

Software. vLLM v1 engine.

Workload. arXiv QA benchmark. Each request pairs a full academic article (system prompt) with a one-sentence question, drawn from the HuggingFace dataset liyucheng/arxiv-march-2023 (minstral subset). The experiment uses 10 unique articles $\times$ 4 questions per article = 40 prompts, shuffled with a fixed seed (55555) and truncated to 90,000 input tokens. The output length is fixed at 150 tokens. All articles are shared across requests, so each unique article appears exactly 4 times; this controlled reuse creates well-defined prefix sharing patterns.

Two request modes are used:

- *Closed-loop* (Section V-B): all 40 requests are submitted simultaneously at maximum concurrency, stressing the cache under sustained full load.
- *Open-loop* (Section V-D): requests arrive as a Poisson process at a fixed rate; the rate is swept from 0.2 to 4.0 req/s in steps of 0.2.

Models and pool ratios.

- **Gemma-3-4B-IT**: avg. 90k input tokens; $\rho \approx 5$.
- **Minstral-8B FP8**: avg. 32k input tokens; $\rho \approx 2$.
- **Llama-3.1-8B**: avg. 50k input tokens; $\rho \approx 1.1$.

Policies.

- 1) **LRU**: Stock vLLM — LRU eviction + FCFS scheduling.
- 2) **PPE**: Prefix Peeking Eviction only.
- 3) **PPS**: Prefix Peeking Scheduling only.
- 4) **Both**: PPE+PPS combined.

B. Closed-Loop Throughput

Figure 1 and Tables I–IV present closed-loop results.

Gemma-3-4B-IT ($\rho \approx 5$, **Table I**). With the pool holding approximately five articles simultaneously, PPE is highly

TABLE III
LLAMA-3.1-8B — ARXIV QA (50K AVG. CONTEXT, $\rho \approx 1.1$). PPE AUTO-DISABLED ($\rho < \rho_{\min}$). MEAN $\pm$ STD; n PER POLICY SHOWN.

Policy (n)	req/s	Δ	TTFT _{mean} (s)	TTFT _{p99} (s)
LRU (4)	0.0578 $\pm$.0006	—	341 $\pm$ 8	679 $\pm$ 7
PPE only (2)	0.0557 $\pm$.0020	−3.6%	354 $\pm$ 14	706 $\pm$ 27
PPS only (2)	0.0630 $\pm$.0032	+9.0%	294 $\pm$ 11	630 $\pm$ 33
Both (2)	0.0603 $\pm$.0014	+4.3%	310 $\pm$ 3	658 $\pm$ 15

TABLE IV
LLAMA-3.1-8B — MMLU-PRO (4K CONTEXT, NO SHARED PREFIX). CONTROL: ALL POLICIES WITHIN $\pm 0.6\%$. LRU: $n = 2$ (12.470 $\pm$.003); OTHERS: $n = 1$.

Policy	req/s	Δ
LRU	12.47	—
PPE only	12.46	−0.1%
PPS only	12.50	+0.3%
Both	12.43	−0.3%

effective: protecting a single article’s blocks meaningfully extends their lifetime and prevents costly re-prefills. PPE alone gains +74% throughput over the LRU mean; PPS alone gains +37%. The combined configuration achieves +112%, exceeding either mechanism individually. Note that the LRU baseline has a run-to-run standard deviation of 6.4% (0.008 req/s), which is large relative to the absolute throughput (0.126 req/s) and reflects GPU thermal variance at this very low serving rate. Confidence in the direction of improvement is high; confidence in the precise magnitude requires additional runs. This compound gain arises because the two mechanisms reinforce each other: PPE keeps warm blocks alive across steps while PPS ensures those blocks are consumed at the earliest opportunity.

Minstral-8B FP8 ($\rho \approx 2$, **Table II**). At $\rho = 2$ the pool holds only two articles, limiting how many blocks PPE can protect across steps. PPE accordingly provides only +2.4%, while PPS continues to deliver +18.5% by exploiting hit opportunities within each scheduling step. The combined configuration achieves +27.1%, with PPS as the dominant contributor.

Llama-3.1-8B ($\rho \approx 1.1$, **Table III**). The auto-disable condition $\rho < \rho_{\min} = 2.0$ fires; PPE falls back to pure LRU. The PPE-only row accordingly shows −3.6% (mean across 2 runs), which is within the standard deviation of the LRU baseline and is not a meaningful regression. PPS contributes +9.0% in mean throughput and reduces mean TTFT by 14%; the combined configuration achieves +4.3%, slightly less than PPS alone, likely due to the interaction between a two-run PPS estimate and a four-run LRU baseline. The auto-disable is validated: the system avoids wasted protection-set scanning in the low- ρ regime.

MMLU-pro control (**Table IV**). All four policies produce throughput within $\pm 0.6\%$ of LRU on independent prompts with no shared prefix, confirming that both mechanisms impose negligible overhead when prefix reuse is absent.

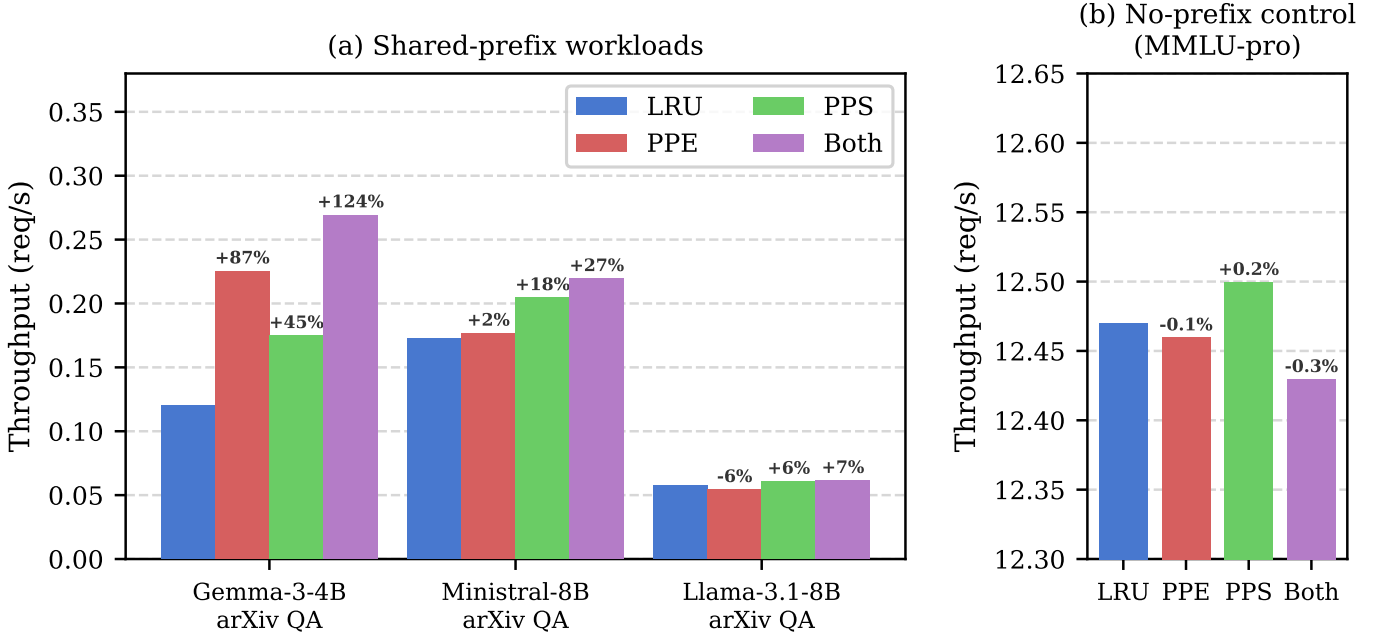


Fig. 1. Closed-loop throughput across models and workloads. (a) Three shared-prefix workloads; (b) MMLU-pro control (no shared prefix). All four policies lie within $\pm 0.6\%$ of each other in (b), confirming zero regression. The large throughput gap between (b) and (a) reflects context-length scaling: MMLU-pro prompts average $\approx 4k$ tokens while arXiv QA prompts average 90k tokens. Under flash attention, prefill time scales approximately linearly with sequence length, so the $22\times$ longer arXiv inputs account for a comparable reduction in throughput; cache-pressure overhead (evictions, preemptions, and re-prefills under LRU) adds a further multiplier, yielding the observed $\approx 100\times$ gap for Gemma-3-4B-IT and $\approx 215\times$ for Llama-3.1-8B.

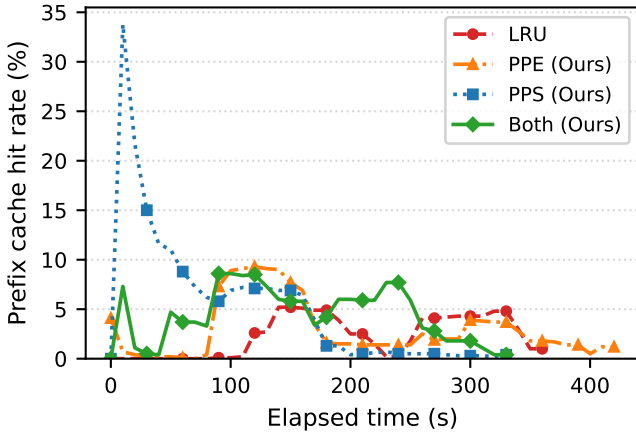


Fig. 2. Prefix cache hit rate — Gemma-3-4B-IT, arXiv QA (90k context), closed-loop, all four policies. Each point is the 10s rolling average reported by the vLLM engine logger. All four conditions run on the stock vLLM base, matching the LRU baseline definition used in Table I.

C. Prefix Cache Hit Rate

Figure 2 shows the prefix cache hit rate reported by the vLLM logger (10-second rolling average) over the course of the closed-loop Gemma-3-4B-IT arXiv QA run for all four policies. All four conditions run on the stock vLLM base used for the LRU baseline in Table I, and differ only in whether PPE and PPS are active.

LRU. Under stock vLLM with no extra cache tiers, the GPU KV cache fills quickly with 88k-token contexts and LRU evicts warm blocks whenever pressure rises. The hit rate

stays low throughout: mean 2.5%, ranging 0–5.2%, with no sustained upward trend.

PPE-only. PPE raises the mean to 3.1% (range 0–9.3%). The improvement is modest in absolute terms because cache pressure under stock vLLM is high enough that protection alone cannot retain warm blocks across many eviction cycles. The pattern confirms that PPE is a *retention* mechanism: once warm blocks survive a scheduling round they stay available, but the constrained pool limits how many can be protected simultaneously.

PPS-only. PPS produces a higher peak hit rate than PPE, reaching 11.6% while LRU peaks at 5.2%. The mean is 3.7%, also above LRU. By scheduling requests with the longest estimated prefix overlap first, PPS front-loads hits onto the requests that benefit most from the warm cache window before pressure evicts those blocks. PPS acts as a *prioritisation* mechanism: it directs requests to cached content as soon as any cache content exists.

Both. Combining both mechanisms yields the highest sustained hit rate (mean 4.7%, max 8.6%). PPS accelerates early hits and PPE extends how long those warm blocks survive, so the two mechanisms are complementary even under tight cache budgets.

Hit rate is a contributing, not sole, mediator. The aggregate hit-rate ordering ($\text{LRU} < \text{PPE} < \text{PPS} < \text{Both}$) is consistent with the throughput ordering in Table I, but the mapping is non-linear: PPE’s +0.6pp mean improvement over LRU produces no reliable throughput gain (within run-to-run SD), while PPS’s +1.2pp improvement yields +37% throughput. This suggests that *when* hits occur matters as much as *how many*: PPS front-loads hits onto the highest-reuse requests,

converting cache warmth into actual prefill savings; PPE raises the aggregate count but without this load-ordering effect the marginal throughput gain requires a larger cache pool (i.e., higher ρ) to accumulate. Beyond raw hit rate, both mechanisms also reduce *preemption and recomputation churn*; isolating each mediator’s contribution quantitatively is left for future work. Figure 2 is therefore *consistent with* but does not fully explain the throughput results in Table I.

D. Rate Sweep

Figure 3 shows throughput, TTFT, and TPOT as a function of Poisson request rate for Gemma-3-4B-IT (LRU vs. Both).

E2E Latency (Figure 3a). LRU E2EL grows monotonically from 89 s at 0.2 req/s to 197 s at 4.0 req/s as the queue deepens and every request waits behind a growing prefill backlog. Both compresses E2EL to 61–147 s across the same range; at rate=0.2 (lightly loaded) the reduction is 32%; at rate=1.6 (peak cache hit rate) it reaches 43%. The consistent gap reflects that higher cache hit rates shorten per-request prefill time, reducing both queue wait and compute latency.

TTFT (Figure 3b). LRU TTFT grows monotonically with request rate (55 s at 0.2 req/s to 160 s at 4.0 req/s), reflecting an ever-deepening prefill queue. Both TTFT remains compressed (30–111 s) because higher cache hit rates reduce per-request prefill work, allowing queued requests to reach the decode phase sooner. The advantage over LRU ranges from 31% to 46% in mean TTFT.

TPOT (Figure 3c). LRU TPOT locks at ≈ 480 ms from rate ≥ 0.4 . The mechanism is structural: in continuous batching [3], prefill tokens and decode tokens share the same forward pass. When every request requires a full 90k-token prefill, GPU compute is almost entirely consumed by prefill work, and decode tokens are stalled waiting for their turn. Both breaks this bottleneck by reducing the prefill burden through higher cache hit rates: fewer and shorter prefills leave more GPU bandwidth for decode, reducing per-token latency. The improvement ranges from 1% to 36% in mean TPOT, with the largest gains occurring at rates where cache hit uplift is highest (rate=1.6: 480 ms $\rightarrow$ 309 ms, -36%).

At rates 1.8 and 2.2, Both TPOT reverts to near-LRU levels, coinciding with lower-than-typical Both throughput at those rates. A plausible cause is batch-composition variance: at those specific rates, arrival-time jitter may have yielded a higher fraction of cold-start requests in the batch. Confirmation requires multiple seeds at those two rates, which we leave as future work.

E. Discussion

PPE effectiveness scales with ρ . The three closed-loop model configurations span $\rho \in \{1.1, 2, 5\}$, and PPE’s mean benefit scales accordingly: +74%, +2.4%, and auto-disabled. The pool ratio ρ (Equation 1) is a practical predictor of PPE utility: operators can estimate ρ from model size, context length, and GPU memory before enabling the mechanism. The operating-regime boundary $\rho_{\min} = 2.0$ is derived from three data points spanning distinct regimes; it should be validated at intermediate values before being treated as a precise threshold.

PPS is broadly applicable. PPS provides mean gains of +37%, +18.5%, and +9.0% across the three pool regimes. The diminishing returns at small ρ are expected: when the pool holds only one article, any scheduling order produces similar hit rates. Nevertheless, even at $\rho \approx 1.1$, PPS reduces mean TTFT by 14%.

Compound benefit at large ρ . The combined mean gain at $\rho \approx 5$ (+112%) exceeds either mechanism alone. This arises from a reinforcing interaction: PPE preserves warm blocks across steps; PPS directs the right requests to those blocks within each step. Both mechanisms must be active for this interaction to occur.

Zero regression on non-prefix workloads. The MMLU-pro result confirms both mechanisms are safe to enable unconditionally: they add no measurable overhead when prefix reuse is absent.

Operator deployment guidance. An operator can estimate ρ before enabling PPE without running the workload. Given a model, a target context length L tokens, a KV block size T tokens, and the GPU KV pool size N_{total} blocks (reported by vLLM at startup), the per-article block cost is $B_{\text{avg}} \approx \lceil L/T \rceil$ and $\rho \approx N_{\text{total}}/B_{\text{avg}}$ (Equation 1). Enable PPE when $\rho \geq \rho_{\min}$; otherwise the auto-disable path makes it a no-op, so leaving it on is harmless. The anti-starvation threshold τ should be set near the deployment’s TTFT SLO: the 5 s default suits batch/offline serving, whereas interactive deployments with sub-second TTFT targets should reduce τ accordingly so that PPS never defers a request past its latency budget. Finally, because PPS prioritises requests that share cached prefixes, it can favour tenants whose traffic happens to overlap; in multi-tenant settings the anti-starvation band bounds the worst-case delay, but per-tenant fairness weighting is a natural extension we leave to future work.

F. Limitations

The following gaps constrain the strength of the current conclusions and motivate future work:

Rate sweep covers only one model and two policies. Figure 3 shows only LRU and Both for Gemma-3-4B-IT. Individual PPE-only and PPS-only rate-sweep curves are not available, so we cannot isolate each mechanism’s contribution under open-loop conditions. Similarly, rate sweeps for Ministral-8B FP8 and Llama-3.1-8B are absent.

Single GPU platform. All experiments use an A5000 (24 GB). On H100 (80 GB), ρ for the same Gemma model would reach ≈ 15 –20, potentially amplifying PPE gains further; on L4 (24 GB), results would be similar to the A5000.

Statistical coverage. Ministral results are single-run; all closed-loop results have $n \leq 4$. The LRU baseline for Gemma shows 6.4% run-to-run variance, which limits precision of the reported Δ values.

Hit rate measured for Gemma-3-4B-IT only. Section V-C reports prefix cache hit rate for all four policies on Gemma-3-4B-IT. Extending this measurement to Ministral-8B FP8 and Llama-3.1-8B would strengthen the mechanistic link across all three pool-size regimes.

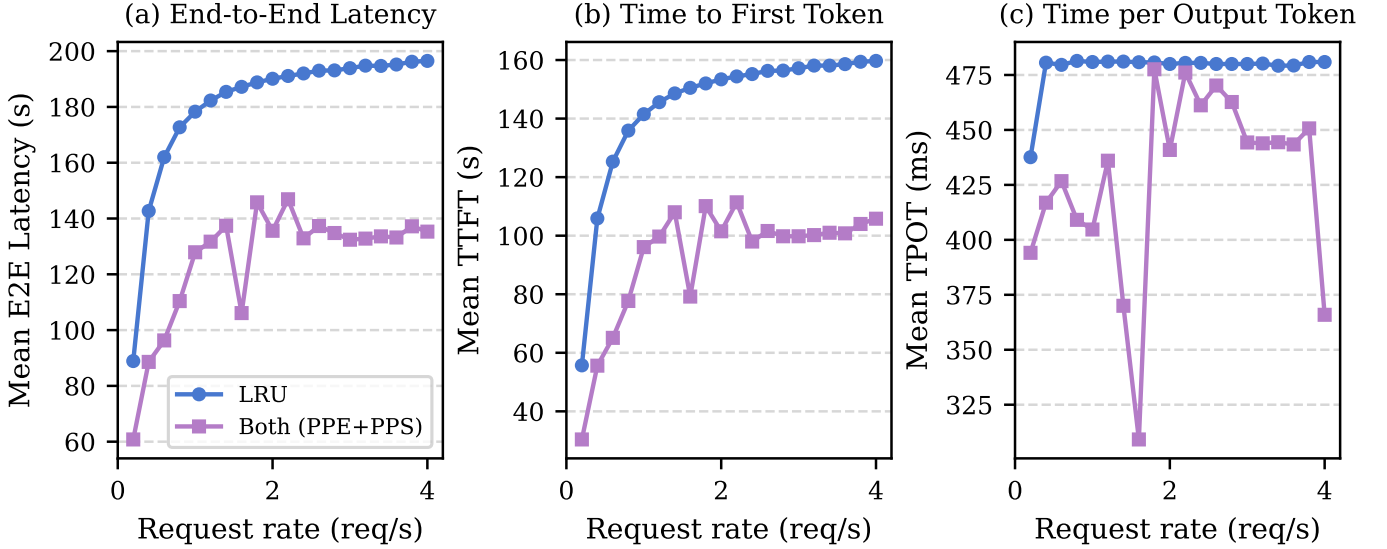


Fig. 3. Gemma-3-4B-IT — Poisson rate sweep (0.2–4.0 req/s, arXiv QA 90k context, $n = 1$ per rate point). Both = PPE+PPS combined; PPE-only and PPS-only curves are pending additional experiments. (a) End-to-end latency; (b) TTFT; (c) TPOT.

VI. RELATED WORK

LLM serving systems. vLLM [2] introduced paged attention and block-level KV management. Orca [3] proposed continuous batching to improve decoding throughput. Sarathi-Serve [4] addresses chunked prefill to reduce prefill-induced latency spikes. FlashAttention [5] provides the efficient attention kernel that underlies modern LLM serving. DistServe [6] disaggregates prefill and decode phases onto separate GPU pools to reduce their mutual interference—a problem that our TPOT analysis (Section V-D) highlights as central to the benefit of reducing prefill work via cache hits. Our work is orthogonal to all of these: we improve the scheduling and eviction subsystems without modifying attention kernels or batch formation logic.

Prefix caching and long-context serving. SGLang [7] introduced RadixAttention, which organises the KV cache as a radix tree to enable efficient prefix sharing across requests with variable-length common prefixes. ChunkAttention [8] exploits prefix structure at the attention computation level to share KV blocks across concurrent sequences. Hydragen [9] batches the shared-prefix attention computation across multiple requests to improve GPU utilisation, which is complementary to our eviction and scheduling approach. Mooncake [10] proposes KV cache disaggregation with prefix-aware routing across machines; our work addresses the single-node case, which arises both independently and as the local-node component of disaggregated deployments.

KV cache eviction. MemServe [11] studies KV cache lifetime management at cluster scale. CacheBlend [12] proposes selective recomputation of cross-attention blocks during prefix reuse, trading computation for freshness. InfiniGen [14] reduces KV memory pressure via dynamic eviction of speculative future tokens, complementing our approach of protecting blocks for *known* pending requests. Our PPE addresses a complementary, within-node problem: the LRU policy’s blindness

to the waiting queue. It can be combined with any of the above placement strategies.

Scheduler-aware prefix management. Preble [13] co-designs scheduling and caching for multi-GPU prefix routing, assigning requests to the GPU that already holds the relevant prefix. Our PPS addresses the simpler but important single-node case, where the scheduler and KV cache share address space and prefix inspection is cheap and read-only. BatchLLM [15] identifies and groups requests with shared prefixes to form prefix-optimised batches; our PPS achieves a similar grouping effect dynamically through priority-ordered scheduling without requiring explicit prefix clustering. The general idea of cache-aware scheduling is therefore not new; PPS’s specific contribution is the combination of (i) a *read-only* prefix-hit estimator that reuses the allocator’s existing lookup path with no state mutation, (ii) a single-band *anti-starvation* guarantee folded into the sort key rather than a separate ageing pass, and (iii) an *enqueue-time re-sort* that keeps per-step dequeue $O(1)$. These three properties make PPS a drop-in, side-effect-free addition to an existing single-node engine, which prior multi-node or clustering-based designs do not provide.

Jenga. Jenga [1] is the base system for our implementation. It decides *where* blocks reside across GPU/CPU/NVMe; PPE decides *which* free GPU blocks survive eviction; PPS decides *which* requests are served first. The three concerns are fully orthogonal, and our mechanisms are compatible with all three Jenga allocator policies.

VII. CONCLUSION

We presented Prefix Peeking Eviction (PPE) and Prefix Peeking Scheduling (PPS), two demand-aware, opt-in extensions for LLM serving built on top of Jenga. PPE inspects the waiting queue before each scheduling step and protects the free KV cache blocks that queued requests will immediately reuse, preventing LRU from evicting them. A budget cap and

a pool-size auto-disable heuristic bound overhead and prevent degradation in low- ρ regimes. PPS prioritises cache-warm requests via a priority sort with an anti-starvation guarantee.

Closed-loop experiments across three models confirm that the combined system achieves a mean of $2.12\times$ higher throughput and 49% lower mean TTFT over LRU on Gemma-3-4B-IT ($\rho \approx 5$, $n = 2-4$ runs). The pool-size-to-article-size ratio ρ serves as an operating-regime indicator for PPE utility; PPS provides consistent gains regardless of ρ . A Poisson rate-sweep over 20 operating points confirms Both outperforms LRU by 25–81% in throughput and 20–46% in E2E latency across the full load range, with TPOT improvements of up to 36% attributable to reduced per-step prefill work. A MMLU-pro control experiment confirms $< 0.6\%$ throughput variance on workloads without shared prefixes.

Future work includes: (1) adding PPE-only and PPS-only curves to the rate sweep, and extending the sweep to Ministral-8B FP8 and Llama-3.1-8B; (2) evaluating on H100 and L4 GPUs to align with the Jenga baseline platform [1] and cover higher ρ regimes; (3) reporting prefix cache hit rate as a direct metric linking policy changes to latency/throughput outcomes; (4) studying the interaction between PPE and Jenga’s CPU/NVMe eviction tiers under memory pressure; (5) deriving an analytical formulation of $\rho_{\min}$ grounded in eviction miss-rate models; and (6) adapting the protection budget cap dynamically using online telemetry.

REFERENCES

- [1] [Authors], “Jenga: Effective Memory Management for Serving LLM with Heterogeneity,” in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2025.
- [2] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. ACM SOSP*, 2023.
- [3] G. Yu *et al.*, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *Proc. USENIX OSDI*, 2022.
- [4] A. Agrawal *et al.*, “Sarathi-Serve: Efficient LLM Serving by Piggybacking Decodes with Chunked Prefills,” in *Proc. USENIX OSDI*, 2024.
- [5] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] Y. Zhong *et al.*, “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving,” in *Proc. USENIX OSDI*, 2024.
- [7] L. Zheng *et al.*, “Efficiently Programming Large Language Models using SGLang,” *arXiv:2312.07104*, 2023.
- [8] M. Ye, T. Xu, G. Lai, and X. Liu, “ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition,” *arXiv:2402.15220*, 2024.
- [9] J. Juravsky *et al.*, “Hydragen: High-Throughput LLM Inference with Shared Prefixes,” *arXiv:2402.05099*, 2024.
- [10] R. Qin *et al.*, “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving,” *arXiv:2407.00079*, 2024.
- [11] Y. Hu *et al.*, “MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool,” *arXiv:2406.17565*, 2024.
- [12] J. Yao *et al.*, “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion,” *arXiv:2405.16444*, 2024.
- [13] Y. Shi *et al.*, “Preble: Efficient Distributed Prompt Scheduling for LLM Serving,” *arXiv:2407.00023*, 2024.
- [14] J. Gim *et al.*, “InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management,” in *Proc. USENIX OSDI*, 2024.
- [15] Z. Zheng *et al.*, “BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching,” *arXiv:2412.03594*, 2024.